

University of West London

Mathematics and Statistics by
Computer
Assignment on MATLAB

Leo Hirani (35114093)
BSc Mathematics and Statistics
27/05/2026.

Table of contents

Abstract.....	3
Chapter	
1.....	4
Chapter	
2.....	5
Chapter 3	6

Chapter	
4.....	7
Conclusion.....	8
Reference.....	8
Appendix	9
Codes.....	9

Abstract

In this assignment, the objective is to write functions for 4 different MATLAB tasks.

Task 1: Create a function that converts a user's input from miles to kilometres.

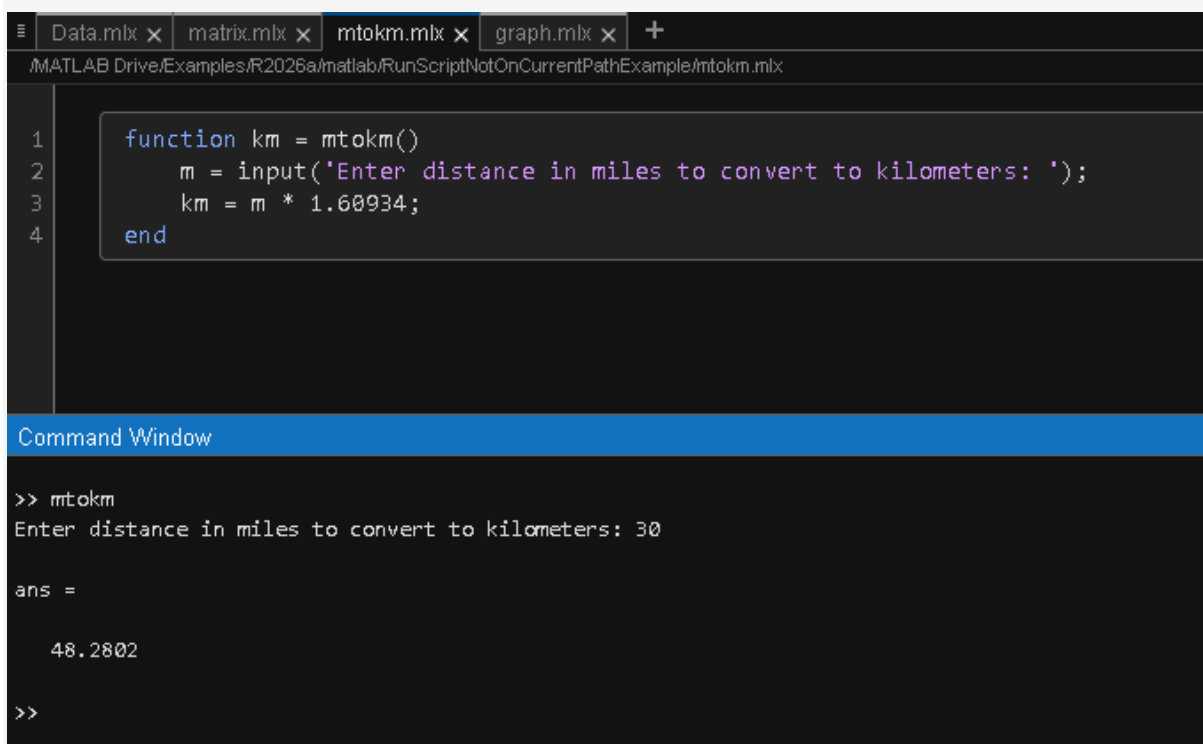
Task 2: Create a function that computes the mean and standard deviation of a set of numbers (N).

Task 3: Write a function that creates a 10x10 matrix (Y) with a mean value of 5 and variance of 9 (standard deviation of 3) and create matrices Y1, Y2, Y3, and Y4 using the Y matrix.

Task 4: Write a function to create graphs for 4 different equations and plot their curves using a single figure.

Chapter 1

The first task is to write a MATLAB function that will convert a numerical input from the user from miles to kilometres.



The image shows a MATLAB environment with a script editor and a command window. The script editor has four tabs: 'Data.mlx', 'matrix.mlx', 'mtokm.mlx', and 'graph.mlx'. The 'mtokm.mlx' tab is active, showing a MATLAB function named 'mtokm'. The function has four lines of code: a function declaration, an input prompt, a calculation, and an end statement. The command window below shows the execution of the function. The user enters '30' in response to the prompt, and the function returns '48.2802'.

```
function km = mtokm()
    m = input('Enter distance in miles to convert to kilometers: ');
    km = m * 1.60934;
end
```

```
>> mtokm
Enter distance in miles to convert to kilometers: 30

ans =

    48.2802

>>
```

<https://www.blackcircles.com/helpcentre/tyres/miles-to-kilometres>

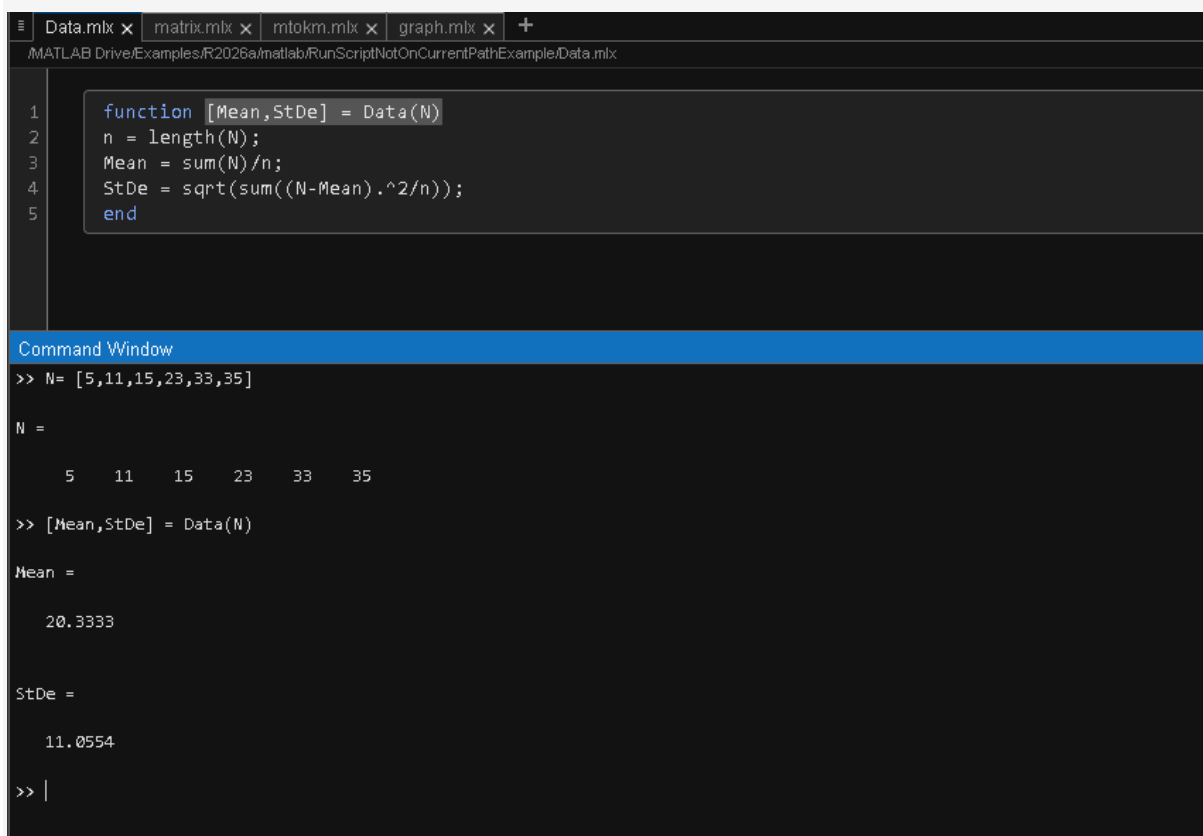
The MATLAB function created for Task 1 was successful, converting 30 miles to 48.2802km, giving its answer to 4 decimal places.

Chapter 2

The second task is to create a MATLAB function that finds the mean and standard deviation of the data of a set of numbers (Data(N)) without using the MATLAB built-in function.

The average value of a data set (the mean) can be calculated via the summation of the numerical values divided by the total numbers.

The spread of the data (the standard deviation) can be calculated by the summation of each data value subtracted from its mean, squared, divided by the total numbers in the data and then square rooted.



The image shows a MATLAB script editor with a function named `Data(N)` that calculates the mean and standard deviation. The function is defined as follows:

```
1 function [Mean,StDe] = Data(N)
2 n = length(N);
3 Mean = sum(N)/n;
4 StDe = sqrt(sum((N-Mean).^2/n));
5 end
```

Below the script editor is the Command Window, which shows the execution of the function. The input data `N` is `[5,11,15,23,33,35]`. The output for the mean is `20.3333` and for the standard deviation is `11.0554`.

```
>> N= [5,11,15,23,33,35]
N =
     5    11    15    23    33    35
>> [Mean,StDe] = Data(N)
Mean =
    20.3333
StDe =
    11.0554
>> |
```

<https://www.calculatorsoup.com/calculators/statistics/mean-median-mode.php>

The MATLAB function successfully gives both the mean and the standard deviation of the numerical data set for any n numbers, giving its accuracy to 4 decimal places.

Chapter 3

The third task is to write a MATLAB function that creates a random 10x10 matrix (Y) with a mean value of 5 and variance of 9, or standard deviation of 3, since the standard deviation is the square root of the variation. The function must also create 4 5x5 matrices (Y1, Y2, Y3 and Y4) using the original matrix Y by using the top left, top right, bottom left and bottom right of the Y matrix. The function must also check the variance and mean of the matrix Y.

```
1 Y = 3.*randn(10,10)+5
2
3 Y1 = Y(1:5,1:5)
4 Y2 = Y(1:5,6:10)
5 Y3 = Y(6:10,1:5)
6 Y4 = Y(6:10,6:10)
7
8 L = numel(Y);
9
10 Mean = sum(Y(:))/L;
11
12 Side = sqrt(sum((Y(:)-Mean).^2)/L);
13
14 MatrixLength = numel(Y);
15 MatrixMean = sum(Y(:))/MatrixLength;
16
17 SideMatrix = sqrt(sum((Y(:)-MatrixMean).^2)/MatrixLength)
```

Y = 10x10

7.9549	3.9006	3.3595	7.8622	8.1847	6.0987	-4.4787	2.6444	-8.8199	5.9583
6.4819	5.2289	-4.2324	10.1395	7.2459	3.0116	8.1284	2.4589	2.2641	6.8227
9.8811	8.7465	8.7415	7.8572	3.4866	2.6474	7.1314	5.8326	2.7838	-8.3289
4.9886	7.3357	3.7486	9.8824	3.8555	-4.9857	9.6296	8.2268	3.6179	-6.4761
4.7817	4.8784	-4.7785	4.7948	6.7505	1.1552	7.6853	-6.5159	-6.4505	10.5661
4.5461	4.3111	3.6531	5.9435	2.4162	10.4669	5.4874	8.4883	2.2329	5.1721
1.3476	9.2853	6.2538	5.2385	-4.2888	6.5115	6.5916	-4.1948	-4.2332	5.5284
11.8975	7.8240	8.2258	3.4332	8.7401	2.7129	5.8697	2.2392	7.9798	8.7289
0.7130	10.1554	1.1712	-8.2574	7.8846	-8.9790	3.6512	5.4375	1.4321	1.1852
4.4280	8.9434	3.4944	8.6288	-4.8126	7.8814	3.8246	7.9194	3.1828	7.5592

Y1 = 5x5

7.9549	3.9006	3.3595	7.8622	8.1847
6.4819	5.2289	-4.2324	10.1395	7.2459
9.8811	8.7465	8.7415	7.8572	3.4866
4.9886	7.3357	3.7486	9.8824	3.8555
4.7817	4.8784	-4.7785	4.7948	6.7505

Y2 = 5x5

6.0987	-4.4787	2.6444	-8.8199	5.9583
3.0116	8.1284	2.4589	2.7641	6.8227
2.6474	7.1314	5.8326	2.7838	-8.3289
-4.9857	9.6296	8.2268	3.6179	-6.4761
1.1552	7.6853	-6.5159	-6.4505	10.5661

Y3 = 5x5

10.4669	5.4874	8.4883	2.2329	5.1721
6.5115	6.5916	-4.1948	-4.2332	5.5284
2.7129	5.8697	2.2392	7.9798	8.7289
3.6512	5.4375	1.4321	1.1852	
3.8246	7.9194	3.1828	7.5592	

Y4 = 5x5

4.5461	4.3111	3.6531	5.9435	2.4162
1.3476	9.2853	6.2538	5.2385	-4.2888
11.8975	7.8240	8.2258	3.4332	8.7401
0.7130	10.1554	1.1712	-8.2574	7.8846
4.4280	8.9434	3.4944	8.6288	-4.8126

Mean = 5.1175
Side = 2.7337
MatrixLength = 100
MatrixMean = 5.1175
SideMatrix = 2.7337

<https://uk.mathworks.com/help/matlab/math/random-numbers-with-specific-mean-and-variance.html>

MATLAB has a built-in random number generator using the normal distribution with mean 0 and standard deviation 1 using 'randn'. Using 'randn' to create a 10x10 matrix with variance of 9 and mean of 5 produces us a Y matrix with these parameters.

Using the in-built function 'name1', the function can find the total number of values in the matrix and find the sum of the total value of all the columns in the matrix using the 2x sum function.

<https://uk.mathworks.com/help/matlab/ref/double.numel.html>

The MATLAB function was a success, outputting a 10x10 matrix and giving 4 subsequent 5x5 matrices using the Y matrix and producing Y1, Y2, Y3 and Y4. The function produced the matrix Y with mean of 5.1175 and variance of 7.47311, checking using the equation for the mean of a matrix and variance of a matrix.

Chapter 4

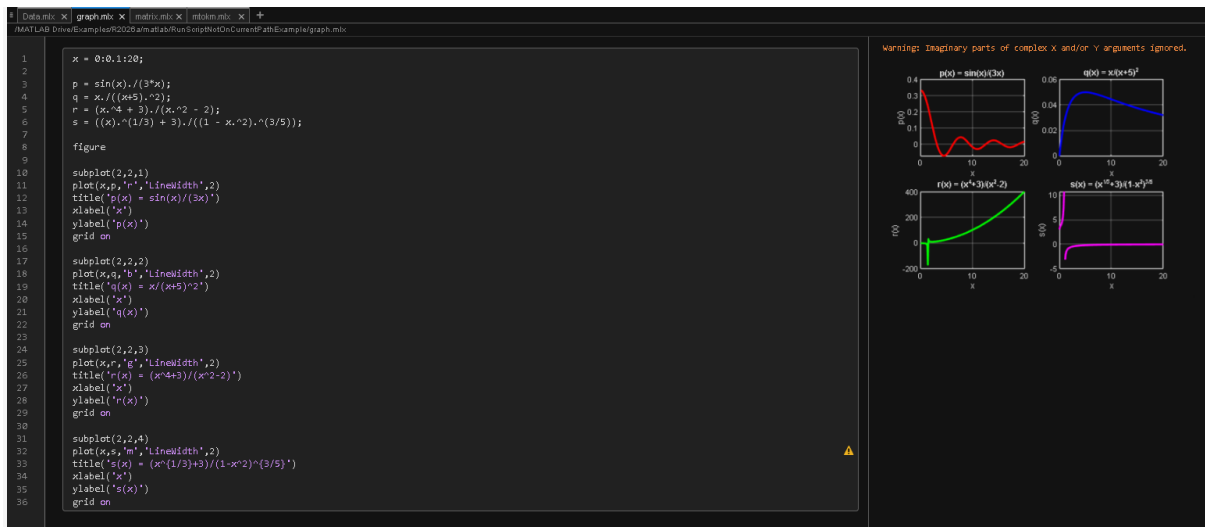
Task 4 is to create a MATLAB function that creates a graph for four equations $p(x)$, $q(x)$, $r(x)$ and $s(x)$ and plots the curves on one figure via 2 rows and 2 columns, labelling the axis and giving values for each tile as the curve progresses from $0 < x < 20$, inclusive of 0 and 20.

i. $p(x) = \frac{\sin x}{3x}$

ii. $q(x) = \frac{x}{(x+5)^3}$

iii. $r(x) = \frac{x^4 + 3}{x^2 - 2}$

iv. $s(x) = \frac{x^{1/3} + 3}{(1 - x^3)^{3/5}}$



the MATLAB function successfully generated 4 graphs, using 2 rows and 2 columns in a single figure for $p(x)$, $q(x)$, $r(x)$ and $s(x)$ accurately.

Conclusion

This assignment was created to demonstrate the use of MATLAB to write four different tasks. Each function has successfully produced the outcome needed for each task and has given accurate results for each of the tasks.

References

<https://www.blackcircles.com/helpcentre/tyres/miles-to-kilometres>

<https://www.calculatorsoup.com/calculators/statistics/mean-median-mode.php>

<https://uk.mathworks.com/help/matlab/math/random-numbers-with-specific-mean-and-variance.html>

<https://uk.mathworks.com/help/matlab/ref/double.numel.html>

Appendix

Codes

Chapter 1:

function km = mtokm()

```
m = input('Enter distance in miles  
to convert to kilometers: ');  
km = m * 1.60934;  
end
```

Chapter 2:

```
function [Mean,StDe] = Data(N)  
n = length(N);  
Mean = sum(N)/n;  
StDe = sqrt(sum((N-Mean).^2/n));  
end
```

Chapter 3:

```
Y = 3.*randn(10,10)+5
```

```
Y1 = Y(1:5,1:5)
```

```
Y2 = Y(1:5,6:10)
```

```
Y3 = Y(6:10,6:10)
```

```
Y4 = Y(6:10,1:5)
```

```
L = numel(Y);
```

```
Mean = sum(Y(:))/L
```



```
StDe = sqrt(sum((Y(:)-Mean).^2)/L)
```

```
MatrixLength = numel(Y)
```

```
MatrixMean =  
sum(Y(:))/MatrixLength
```

```
StDeMatrix = sqrt(sum((Y(:)-  
MatrixMean).^2)/MatrixLength)
```

Chapter 4:

```
x = 0:0.1:20;
```

```
p = sin(x)./(3*x);
```

```
q = x./((x+5).^2);
```

```
r = (x.^4 + 3)./(x.^2 - 2);
```

```
s = ((x).^(1/3) + 3)./((1 -  
x.^2).^(3/5));
```

```
figure
```

```
subplot(2,2,1)
```

```
plot(x,p,'r','LineWidth',2)
```

```
title('p(x) = sin(x)/(3x)')
```

```
xlabel('x')
```

```
ylabel('p(x)')
```

```
grid on
```

```
subplot(2,2,2)
```

```
plot(x,q,'b','LineWidth',2)
```

```
title('q(x) = x/(x+5)^2')
```

```
xlabel('x')
```

```
ylabel('q(x)')
```

```
grid on
```

```
subplot(2,2,3)
```

```
plot(x,r,'g','LineWidth',2)
```

```
title('r(x) = (x^4+3)/(x^2-2)')
```

```
xlabel('x')
```

```
ylabel('r(x)')
```

```
grid on
```

```
subplot(2,2,4)
```

```
plot(x,s,'m','LineWidth',2)
```

```
title('s(x) = (x^{1/3}+3)/(1-x^2)^{3/5}')
```

```
xlabel('x')
```

```
ylabel('s(x)')
```

grid on